CHAPTER 9

NumPy for Numerical Computing

Python Training — Instructor Guide & Study Material

Topics Covered

9.1 Introduction to NumPy
9.2 Creating Arrays
9.3 Array Operations
9.4 Indexing and Slicing
9.5 Array Manipulation
9.6 Linear Algebra with NumPy
9.7 Reading and Writing Data
9.8 Lab: NumPy in Practice

Section 9.1

Introduction to NumPy

Why NumPy? Memory, speed, and the scientific Python stack

9.1.1 Why NumPy? Performance vs Pure Python

NumPy (Numerical Python) is a fundamental library for scientific computing in Python. Before understanding what NumPy is, it is important to understand why it exists.

Python is a dynamically-typed, interpreted language. While this makes Python flexible and easy to write, it introduces significant overhead for numerical computation:

- Every Python object carries metadata — type information, reference count, memory address.
- A Python list can hold mixed types (integers, strings, floats all in one list), which is flexible but expensive.
- Iterating over a Python list involves a Python-level loop — the interpreter processes each step one at a time.
- There is no concept of 'vectorized' math in pure Python — you need explicit for loops.

Consider a simple example — doubling every element in a list:

```
# Pure Python
nums = [1, 2, 3, 4, 5]
doubled = [x * 2 for x in nums]   # Python loop, slow for large data

# NumPy
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
doubled = arr * 2                  # Vectorized: no Python loop needed
```

The NumPy version works at C speed, calling compiled, optimized code under the hood. For large arrays (millions of elements), NumPy can be 50 to 200 times faster than equivalent pure Python code.

TIP: NumPy's speed comes from two things: (1) data stored in contiguous memory as a C array, and (2) operations implemented in compiled C/Fortran code. Python is only involved in dispatching the call.

Performance benchmark — summing 10 million numbers:

```
import time, numpy as np

# Pure Python
big_list = list(range(10_000_000))
t0 = time.time()
total = sum(big_list)
print(f'Python list: {time.time() - t0:.3f} s')   # ~0.25 s

# NumPy
```

```
arr = np.arange(10_000_000)
t0 = time.time()
total = arr.sum()
print(f'NumPy array: {time.time() - t0:.3f} s')    # ~0.01 s  (25x faster)
```

9.1.2 NumPy vs Python Lists — Memory and Speed

Understanding the memory difference helps explain NumPy's performance advantage.

Feature	Python List	NumPy Array
Element types	Mixed (any Python object)	Homogeneous (same dtype)
Memory layout	Array of pointers to objects	Contiguous C array
Memory per int	~28 bytes per element	~8 bytes (int64)
Arithmetic	Explicit Python for-loop	Vectorized (C speed)
Broadcasting	Not supported	Built-in
N-dimensional	Nested lists manually	Native ndarray
Math functions	import math; manual loops	np.sqrt, np.exp, etc.

Memory comparison for 1 million integers:

```
import sys, numpy as np

py_list = list(range(1_000_000))
np_arr  = np.arange(1_000_000, dtype=np.int64)

# Python list: each int object ~28 bytes + 8 bytes for pointer = ~36 MB
# NumPy int64 array: 8 bytes per element = ~8 MB

print(sys.getsizeof(py_list))          # ~8 MB (just the pointer array!)
print(np_arr.nbytes)                   # 8,000,000 bytes = ~8 MB
# The list elements themselves add another ~28 MB in Python heap
```

NOTE: Python's getsizeof() only measures the list container (pointer array), not the objects themselves. The actual memory for Python list elements is typically 3-4x higher than the NumPy equivalent.

9.1.3 Installing and Importing NumPy

Installation is straightforward using pip or conda:

```
# Using pip
pip install numpy

# Using conda (Anaconda/Miniconda)
```

```
conda install numpy

# Install specific version
pip install numpy==1.26.4

# Verify installation
python -c "import numpy as np; print(np.__version__)"
```

The universal import convention used throughout the Python community is:

```
import numpy as np

# This alias 'np' is universal — every tutorial, book,
# and production codebase uses this convention.

arr = np.array([1, 2, 3])
print(np.__version__)   # e.g., '1.26.4'
```

TIP: Always use 'import numpy as np'. Never use 'from numpy import *' — this pollutes your namespace and creates conflicts with built-in names like sum and min.

NumPy is the foundation of the entire Python scientific stack:

- pandas — DataFrame is built on NumPy arrays
- scikit-learn — all ML algorithms take and return NumPy arrays
- TensorFlow / PyTorch — have NumPy-compatible APIs
- Matplotlib — plots NumPy arrays natively
- SciPy — extends NumPy with advanced scientific algorithms

Section 9.2

Creating Arrays

np.array, zeros, ones, arange, linspace, random, and array attributes

9.2.1 np.array(), np.zeros(), np.ones(), np.empty()

np.array() — Converting Python data to ndarray

The most basic way to create an array is from an existing Python list or tuple:

```
import numpy as np

# 1D array from a list
a = np.array([10, 20, 30, 40, 50])
print(a)           # [10 20 30 40 50]
print(type(a))     # <class 'numpy.ndarray'>
print(a.dtype)     # int64

# Specify dtype explicitly
b = np.array([1.0, 2.5, 3.7], dtype=np.float32)
print(b.dtype)     # float32

# 2D array from nested list (matrix)
mat = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
print(mat.shape)   # (3, 3)
print(mat.ndim)    # 2

# 3D array
arr3d = np.array([[[1,2],[3,4]], [[5,6],[7,8]]])
print(arr3d.shape) # (2, 2, 2)
```

np.zeros(), np.ones(), np.empty(), np.full()

These functions create arrays of a specified shape filled with constant values:

```
# np.zeros — filled with 0.0 (float64 by default)
z = np.zeros((3, 4))          # 3 rows, 4 columns
print(z)
# [[0. 0. 0. 0.]
#  [0. 0. 0. 0.]
#  [0. 0. 0. 0.]]

# np.ones — filled with 1.0
o = np.ones((2, 3))
print(o)
# [[1. 1. 1.]
#  [1. 1. 1.]]

# np.empty — uninitialized (random garbage from memory!)
```

```
# Fastest to create — use when you will overwrite every element
e = np.empty((2, 2))

# np.full — fill with a specific constant
f = np.full((2, 3), 7)
print(f)
# [[7 7 7]
#  [7 7 7]]

# np.eye — identity matrix (1s on diagonal)
identity = np.eye(3)
print(identity)
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]

# Specify integer dtype
iz = np.zeros((3,), dtype=np.int32)
print(iz)   # [0 0 0]  (no decimal points)
```

⚠ **WARNING:** np.empty() does NOT fill with zeros — it returns whatever was already in memory. Values are unpredictable. Only use it when you are about to assign values to every element immediately.

9.2.2 np.arange() and np.linspace()

np.arange() — Integer and Float Ranges

np.arange() works like Python's range() but returns an ndarray:

```
# np.arange(stop)
np.arange(5)           # [0 1 2 3 4]

# np.arange(start, stop)
np.arange(1, 6)        # [1 2 3 4 5]

# np.arange(start, stop, step)
np.arange(0, 10, 2)    # [0 2 4 6 8]
np.arange(10, 0, -2)   # [10  8  6  4  2]

# Float step (use with caution!)
np.arange(0.0, 1.0, 0.25)   # [0.   0.25 0.5  0.75]
```

⚠ **WARNING:** Avoid np.arange() with float steps — floating point rounding can cause unexpected extra or missing values. Use np.linspace() instead when you need a specific number of points over a float range.

np.linspace() — Fixed Number of Points

np.linspace() creates a specified number of evenly-spaced points between a start and stop value. Both endpoints are included by default:

```
# np.linspace(start, stop, num)
np.linspace(0, 1, 5)
# [0.   0.25 0.5  0.75 1.  ]  — 5 points, endpoints included

np.linspace(0, 10, 11)
# [ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]

# Exclude the endpoint
np.linspace(0, 1, 5, endpoint=False)
# [0.  0.2 0.4 0.6 0.8]

# Common use: smooth curve for plotting
x = np.linspace(0, 2 * np.pi, 300)
y = np.sin(x)    # 300 points for a smooth sine curve

# Get step size along with the array
points, step = np.linspace(0, 1, 5, retstep=True)
print(step)      # 0.25
```

TIP: Use np.arange() when you know the step size (e.g., every 0.5 units). Use np.linspace() when you know how many points you want (e.g., 100 points for plotting).

9.2.3 Random Arrays

NumPy's random module provides functions for creating arrays with random values from various distributions:

```
import numpy as np

# Uniform distribution [0.0, 1.0)
np.random.rand(3)           # 1D: [0.42, 0.81, 0.17]
np.random.rand(2, 4)        # 2D: 2x4 matrix of floats in [0,1)

# Standard normal distribution (mean=0, std=1)
np.random.randn(5)          # [-1.2, 0.3, 1.8, -0.1, 0.9]
np.random.randn(3, 3)       # 3x3 matrix

# Random integers
np.random.randint(1, 7)          # one die roll: int in [1,6]
np.random.randint(1, 7, size=5)  # five dice: [3, 1, 6, 2, 4]
np.random.randint(0, 100, (3,3)) # 3x3 matrix of ints in [0,99]

# Random floats from uniform [low, high)
np.random.uniform(low=5.0, high=10.0, size=4)

# Fix the random seed for reproducibility
np.random.seed(42)
np.random.rand(3)   # Always produces same values: [0.374, 0.951, 0.732]
```

Modern NumPy recommends using the Generator API for random numbers (NumPy 1.17+):

```
# Modern approach — Generator API
rng = np.random.default_rng(seed=42)   # Create a Generator
```

```
rng.random(5)             # Uniform [0, 1)
rng.standard_normal(5)    # Standard normal
rng.integers(1, 7, 5)     # Random integers
rng.uniform(5, 10, 5)     # Uniform [5, 10)

# Advantages: better statistical properties, reproducible, thread-safe
```

> **TIP:** Always set a random seed (np.random.seed() or rng = np.random.default_rng(seed=42)) when writing code that others need to reproduce — in notebooks, tests, or tutorials.

9.2.4 Array Attributes — shape, dtype, ndim, size

Every NumPy array has a set of attributes that describe its structure:

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

print(arr.shape)    # (2, 3)   — 2 rows, 3 columns
print(arr.ndim)     # 2        — number of dimensions
print(arr.size)     # 6        — total number of elements
print(arr.dtype)    # int64    — data type of elements
print(arr.nbytes)   # 48       — total bytes in memory (6 × 8 bytes)
print(arr.itemsize) # 8        — bytes per element

# 1D array
v = np.array([10.0, 20.0, 30.0])
print(v.shape)   # (3,)   — tuple with one element for 1D
print(v.ndim)    # 1

# 3D array
t = np.zeros((2, 3, 4))
print(t.shape)   # (2, 3, 4)
print(t.size)    # 24  (2 * 3 * 4)
```

Attribute	Description	Example Output
arr.shape	Tuple: (rows, cols, ...)	(3, 4)
arr.ndim	Number of dimensions	2
arr.size	Total element count	12
arr.dtype	Element data type	float64
arr.nbytes	Total bytes in memory	96
arr.itemsize	Bytes per single element	8
arr.T	Transposed view	shape (4, 3)

Section 9.3

Array Operations

Element-wise arithmetic, broadcasting, math functions, and aggregation

9.3.1 Element-wise Arithmetic Operations

NumPy arrays support arithmetic operators that work element-by-element. No loops are needed:

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])

# Element-wise operations
print(a + b)     # [11 22 33 44 55]
print(b - a)     # [ 9 18 27 36 45]
print(a * b)     # [ 10  40  90 160 250]
print(b / a)     # [10. 10. 10. 10. 10.]
print(a ** 2)    # [ 1  4  9 16 25]
print(b % 3)     # [1 2 0 1 2]  (modulo)

# Scalar operations (applied to every element)
print(a + 100)    # [101 102 103 104 105]
print(a * 2.5)    # [ 2.5  5.   7.5 10.  12.5]
print(b / 10)     # [ 1.  2.  3.  4.  5.]

# 2D matrix operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A + B)    # [[ 6  8] [10 12]]
print(A * B)    # [[ 5 12] [21 32]]  # element-wise, NOT matrix multiply
```

⚠ **WARNING:** arr1 * arr2 in NumPy is element-wise multiplication, NOT matrix multiplication. For matrix multiplication use np.dot(A, B) or A @ B.

9.3.2 Broadcasting Rules and Examples

Broadcasting is NumPy's mechanism for performing arithmetic between arrays of different shapes. Instead of requiring identical shapes, NumPy automatically 'stretches' the smaller array to match the larger one.

Broadcasting Rules (applied in order):

1. If arrays have different number of dimensions, prepend 1s to the shape of the smaller array.
2. Arrays with size 1 along a dimension are stretched to match the other array's size in that dimension.
3. Arrays must either have equal sizes or size 1 in each dimension, or broadcasting fails.

```
import numpy as np

# Example 1: scalar broadcast
arr = np.array([1, 2, 3, 4])
print(arr + 10)   # [11 12 13 14]  — 10 broadcast to shape (4,)

# Example 2: 2D + 1D
A = np.array([[1, 2, 3],
              [4, 5, 6]])    # shape (2, 3)
row = np.array([10, 20, 30])  # shape (3,)  → treated as (1, 3)

print(A + row)
# [[11 22 33]
#  [14 25 36]]
# row is added to EACH row of A

# Example 3: column vector broadcast
col = np.array([[100],
                [200]])      # shape (2, 1)
print(A + col)
# [[101 102 103]
#  [204 205 206]]
# col is added to EACH column of A

# Example 4: broadcast in both dimensions
row  = np.array([1, 2, 3])   # shape (3,)
col  = np.array([[10], [20], [30]])  # shape (3, 1)
print(row + col)
# [[11 12 13]
#  [21 22 23]
#  [31 32 33]]
```

NOTE: Broadcasting never copies data. The array is conceptually stretched but NumPy performs this without allocating new memory. This is both fast and memory-efficient.

9.3.3 Mathematical Functions

NumPy provides universal functions (ufuncs) that operate element-wise on arrays:

```
import numpy as np

arr = np.array([1, 4, 9, 16, 25], dtype=float)

# Square root
print(np.sqrt(arr))    # [1. 2. 3. 4. 5.]

# Exponential (e^x)
print(np.exp(np.array([0, 1, 2])))
# [1.         2.71828183 7.3890561 ]

# Natural logarithm
print(np.log(arr))     # [0.    1.386 2.197 2.773 3.219]
print(np.log2(arr))    # [0.    2.    3.170 4.    4.644]
```

```
print(np.log10(arr))   # [0.    0.602 0.954 1.204 1.398]

# Trigonometric functions
angles = np.array([0, np.pi/6, np.pi/4, np.pi/3, np.pi/2])
print(np.sin(angles))  # [0.    0.5   0.707 0.866 1.   ]
print(np.cos(angles))  # [1.    0.866 0.707 0.5   0.   ]
print(np.tan(angles))  # [0.    0.577 1.    1.732  ∞   ]

# Rounding
x = np.array([1.3, 2.7, -1.5, 3.5])
print(np.floor(x))     # [ 1.  2. -2.  3.]
print(np.ceil(x))      # [ 2.  3. -1.  4.]
print(np.round(x))     # [ 1.  3. -2.  4.]
print(np.abs(x))       # [1.3 2.7 1.5 3.5]
```

9.3.4 Aggregation Functions

Aggregation functions reduce an array to a single value (or aggregate along an axis):

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

# Global aggregations (reduce entire array)
print(arr.sum())      # 45
print(arr.mean())     # 5.0
print(arr.std())      # 2.582
print(arr.var())      # 6.666
print(arr.min())      # 1
print(arr.max())      # 9
print(arr.prod())     # 362880  (product of all)

# Axis-specific aggregations
# axis=0: collapse rows (operate down columns)
print(arr.sum(axis=0))   # [12 15 18]  — column sums
print(arr.mean(axis=0))  # [4. 5. 6.]  — column means

# axis=1: collapse columns (operate across rows)
print(arr.sum(axis=1))   # [ 6 15 24]  — row sums
print(arr.mean(axis=1))  # [2. 5. 8.]  — row means

# Cumulative sum
v = np.array([1, 2, 3, 4, 5])
print(np.cumsum(v))      # [ 1  3  6 10 15]
print(np.cumprod(v))     # [  1   2   6  24 120]

# Index of min/max
print(arr.argmin())      # 0  (flat index)
print(arr.argmax())      # 8  (flat index)
print(arr.argmin(axis=0))  # [0 0 0]  — row index of min per column
```

> **TIP:** Use arr.sum(axis=0) to get column-wise totals (think: 'collapse down the rows'), and arr.sum(axis=1) for row-wise totals (collapse across columns). The axis number refers to the dimension that gets collapsed.

Section 9.4

Indexing and Slicing

Basic, boolean, fancy indexing, and multi-dimensional slicing

9.4.1 Basic Indexing — 1D and 2D

```
import numpy as np

# ── 1D Indexing ──
arr = np.array([10, 20, 30, 40, 50])
#                0   1   2   3   4   (positive indices)
#               -5  -4  -3  -2  -1   (negative indices)

print(arr[0])     # 10   — first element
print(arr[-1])    # 50   — last element
print(arr[-2])    # 40   — second-to-last

# Slicing: arr[start:stop:step]  (stop is exclusive)
print(arr[1:4])   # [20 30 40]
print(arr[:3])    # [10 20 30]   — from beginning
print(arr[2:])    # [30 40 50]   — to end
print(arr[::2])   # [10 30 50]   — every other element
print(arr[::-1])  # [50 40 30 20 10]  — reversed

# ── 2D Indexing ──
mat = np.array([[11, 12, 13],
                [21, 22, 23],
                [31, 32, 33]])

print(mat[0, 0])   # 11  — row 0, col 0
print(mat[1, 2])   # 23  — row 1, col 2
print(mat[-1, -1]) # 33  — last row, last column

# Selecting a full row
print(mat[0])        # [11 12 13]  — entire row 0
print(mat[0, :])     # [11 12 13]  — same

# Selecting a full column
print(mat[:, 1])     # [12 22 32]  — entire column 1

# Sub-matrix slicing
print(mat[0:2, 1:3]) # [[12 13] [22 23]]  — rows 0-1, cols 1-2
```

NOTE: NumPy slices return VIEWS, not copies! Modifying the slice modifies the original array. Use .copy() if you need an independent copy: sub = arr[1:3].copy()

9.4.2 Boolean Indexing

Boolean indexing uses a boolean array (True/False) as an index to select elements. This is extremely powerful for filtering data:

```
import numpy as np

scores = np.array([45, 82, 60, 91, 73, 55, 88, 37])

# Create a boolean mask
mask = scores >= 70
print(mask)
# [False  True False  True  True False  True False]

# Use the mask to select elements
print(scores[mask])   # [82 91 73 88]

# One-liner: combine condition and indexing
print(scores[scores >= 70])   # [82 91 73 88]
print(scores[scores < 60])    # [45 55 37]

# Combined conditions (use & for AND, | for OR)
print(scores[(scores >= 60) & (scores < 80)])  # [82 73 88]... wait
print(scores[(scores >= 60) & (scores < 80)])  # [60 73]

# Assign values using boolean mask
scores[scores < 60] = 60   # Set all failing scores to 60 (floor)
print(scores)  # [60 82 60 91 73 60 88 60]

# np.where — vectorized if-else
grade = np.where(scores >= 70, 'Pass', 'Fail')
print(grade)
# ['Fail' 'Pass' 'Fail' 'Pass' 'Pass' 'Fail' 'Pass' 'Fail']
```

⚠ **WARNING:** Use & (bitwise AND) and | (bitwise OR) for combining NumPy boolean arrays, NOT 'and'/'or' keywords. Also, always wrap each condition in parentheses: (a > 1) & (a < 5).

9.4.3 Fancy Indexing with Integer Arrays

Fancy indexing lets you select elements at arbitrary positions by providing an array of indices:

```
import numpy as np

arr = np.array([100, 200, 300, 400, 500, 600])

# Select elements at indices 0, 2, 4
idx = [0, 2, 4]
print(arr[idx])   # [100 300 500]

# Repeat indices (selects same element multiple times)
print(arr[[1, 1, 3, 0]])  # [200 200 400 100]

# 2D fancy indexing
```

```
mat = np.array([[10, 20, 30],
                [40, 50, 60],
                [70, 80, 90]])

# Select rows 0 and 2
print(mat[[0, 2]])
# [[10 20 30]
#  [70 80 90]]

# Select specific (row, col) pairs: (0,1), (1,2), (2,0)
rows = [0, 1, 2]
cols = [1, 2, 0]
print(mat[rows, cols])   # [20 60 70]

# Sorting by index (argsort returns indices that would sort array)
data = np.array([30, 10, 50, 20, 40])
sorted_idx = np.argsort(data)
print(sorted_idx)           # [1 3 0 4 2]
print(data[sorted_idx])     # [10 20 30 40 50]  — sorted
```

NOTE: Unlike boolean indexing, fancy indexing always returns a COPY, not a view. Modifying the result does not affect the original array.

9.4.4 Multi-dimensional Slicing

```
import numpy as np

# 3D array: think of it as 2 'sheets', each 3 rows × 4 cols
arr3d = np.arange(24).reshape(2, 3, 4)
print(arr3d)
# [[[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]]
#
#  [[12 13 14 15]
#   [16 17 18 19]
#   [20 21 22 23]]]

# First sheet (index 0 along first axis)
print(arr3d[0])
# [[ 0  1  2  3]  [ 4  5  6  7]  [ 8  9 10 11]]

# Element at sheet 1, row 2, col 3
print(arr3d[1, 2, 3])   # 23

# All sheets, row 1, all columns
print(arr3d[:, 1, :])
# [[ 4  5  6  7]
#  [16 17 18 19]]

# Using np.newaxis to add dimensions
v = np.array([1, 2, 3])
```

```
print(v.shape)              # (3,)
print(v[np.newaxis, :].shape)  # (1, 3) — row vector
print(v[:, np.newaxis].shape)  # (3, 1) — column vector

# Ellipsis (...) selects all remaining dimensions
print(arr3d[0, ...])   # same as arr3d[0, :, :]
print(arr3d[..., 2])   # all sheets, all rows, column 2
```

Section 9.5

Array Manipulation

Reshape, stack, split, and transpose arrays

9.5.1 Reshaping — reshape(), flatten(), ravel()

```
import numpy as np

arr = np.arange(12)   # [ 0  1  2  3  4  5  6  7  8  9 10 11]

# reshape — change shape without changing data
mat = arr.reshape(3, 4)   # 3 rows, 4 columns
print(mat)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

# Use -1 to let NumPy compute one dimension automatically
arr.reshape(4, -1)    # shape (4, 3) — NumPy computes 3
arr.reshape(-1, 6)    # shape (2, 6) — NumPy computes 2
arr.reshape(2, 2, 3)  # 3D: shape (2, 2, 3)

# flatten() — always returns a COPY as 1D
flat = mat.flatten()
flat[0] = 999
print(mat[0, 0])   # 0  — original unchanged

# ravel() — returns 1D view (no copy, if possible)
flat2 = mat.ravel()
flat2[0] = 999
print(mat[0, 0])   # 999  — original CHANGED (it's a view!)

# Reshape to column/row vector
v = np.array([1, 2, 3])
col = v.reshape(-1, 1)   # shape (3, 1) — column vector
row = v.reshape(1, -1)   # shape (1, 3) — row vector
```

⚠ **WARNING:** reshape() returns a VIEW when possible (no memory copy). ravel() also returns a view. flatten() always returns a copy. If you modify a view, the original changes too.

9.5.2 Stacking Arrays

```
import numpy as np

a = np.array([[1, 2], [3, 4]])   # shape (2, 2)
b = np.array([[5, 6], [7, 8]])   # shape (2, 2)

# np.vstack — stack vertically (along rows)
print(np.vstack([a, b]))
```

```
# [[1 2] [3 4] [5 6] [7 8]]   shape (4, 2)

# np.hstack — stack horizontally (along columns)
print(np.hstack([a, b]))
# [[1 2 5 6] [3 4 7 8]]   shape (2, 4)

# np.concatenate — general stacking along any axis
np.concatenate([a, b], axis=0)   # same as vstack: shape (4, 2)
np.concatenate([a, b], axis=1)   # same as hstack: shape (2, 4)

# np.stack — creates a NEW axis
result = np.stack([a, b], axis=0)   # shape (2, 2, 2)
result = np.stack([a, b], axis=2)   # shape (2, 2, 2)

# 1D array stacking
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
np.hstack([x, y])    # [1 2 3 4 5 6]   — concatenate
np.vstack([x, y])    # [[1 2 3] [4 5 6]]  — 2D matrix
np.column_stack([x, y])  # [[1 4] [2 5] [3 6]]  — as columns
```

9.5.3 Splitting Arrays

```
import numpy as np

arr = np.arange(12).reshape(3, 4)
print(arr)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

# np.split — split into N equal parts along axis
parts = np.split(arr, 3, axis=0)   # Split into 3 rows
# [array([[0,1,2,3]]), array([[4,5,6,7]]), array([[8,9,10,11]])]

# np.hsplit — split horizontally (along columns)
left, right = np.hsplit(arr, 2)    # Two halves
print(left)    # [[ 0  1] [ 4  5] [ 8  9]]
print(right)   # [[ 2  3] [ 6  7] [10 11]]

# np.vsplit — split vertically (along rows)
top, bottom = np.vsplit(arr, [1])   # First 1 row vs rest

# Split at specific indices
v = np.arange(10)
a, b, c = np.split(v, [3, 7])   # [0:3], [3:7], [7:]
print(a)   # [0 1 2]
print(b)   # [3 4 5 6]
print(c)   # [7 8 9]
```

9.5.4 Transposing

```
import numpy as np

mat = np.array([[1, 2, 3],
                [4, 5, 6]])
print(mat.shape)    # (2, 3)

# .T attribute — transpose
T = mat.T
print(T)
# [[1 4]
#  [2 5]
#  [3 6]]
print(T.shape)      # (3, 2)

# np.transpose() — same as .T for 2D; more control for nD
np.transpose(mat)   # [[1 4] [2 5] [3 6]]

# Transpose is a view — no memory copy
T[0, 0] = 999
print(mat[0, 0])    # 999 — original changed!

# 3D transpose with axis permutation
arr3d = np.zeros((2, 3, 4))
print(np.transpose(arr3d, (2, 0, 1)).shape)  # (4, 2, 3)

# Common use: converting row vector to column vector
row = np.array([[1, 2, 3]])   # shape (1, 3)
col = row.T                   # shape (3, 1)
```

Section 9.6

Linear Algebra with NumPy

Matrix multiplication, determinants, inverse, eigenvalues, and solving systems

9.6.1 Matrix Multiplication — np.dot() and @ Operator

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

# np.dot() — matrix multiplication (dot product)
C = np.dot(A, B)
print(C)
# [[19 22]
#  [43 50]]

# @ operator — same as np.dot() for 2D arrays (Python 3.5+)
C = A @ B
print(C)   # same result

# Matrix × vector
v = np.array([1, 2])
print(A @ v)   # [5 11]  — [1*1+2*2, 3*1+4*2]

# Dot product of two 1D vectors
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
print(np.dot(x, y))   # 32  (1*4 + 2*5 + 3*6)

# Batch matrix multiplication (3D arrays)
batch_A = np.random.rand(10, 3, 4)  # 10 matrices of shape (3,4)
batch_B = np.random.rand(10, 4, 5)  # 10 matrices of shape (4,5)
result = batch_A @ batch_B           # result shape: (10, 3, 5)
```

TIP: Use the @ operator for matrix multiplication in modern Python (3.5+). It is cleaner and more readable than np.dot(). Remember: A * B is element-wise, A @ B is matrix multiplication.

9.6.2 np.linalg Module — det, inv, solve

```
import numpy as np
from numpy import linalg as LA

A = np.array([[2, 1],
              [5, 3]])
```

```
# Determinant
det = LA.det(A)
print(f'Determinant: {det}')   # 1.0

# Inverse
A_inv = LA.inv(A)
print('Inverse:')
print(A_inv)
# [[ 3. -1.]
#  [-5.  2.]]

# Verify: A × A_inv should be identity matrix
print(np.round(A @ A_inv, 10))
# [[1. 0.]
#  [0. 1.]]

# Rank of a matrix
print(LA.matrix_rank(A))   # 2 (full rank)

# Norms
print(LA.norm(A))          # Frobenius norm by default
print(LA.norm(A, ord=1))   # Max column sum
print(LA.norm(A, ord=2))   # Spectral norm (largest singular value)

# Singular Value Decomposition
U, S, Vt = LA.svd(A)
print('Singular values:', S)
```

9.6.3 Solving Linear Systems — np.linalg.solve

Solving a system of linear equations Ax = b, where A is the coefficient matrix and b is the right-hand side:

```
import numpy as np
from numpy import linalg as LA

# Solve the system:
#   2x + y  = 5
#   5x + 3y = 12

A = np.array([[2, 1],
              [5, 3]])
b = np.array([5, 12])

x = LA.solve(A, b)
print(x)   # [3. -1.]  →  x=3, y=-1

# Verify: A @ x should equal b
print(np.allclose(A @ x, b))   # True

# Note: solve() is FASTER and MORE NUMERICALLY STABLE than inv()
# Avoid: x = LA.inv(A) @ b   — don't do this in practice!
# Prefer: x = LA.solve(A, b)
```

```
# Solve multiple right-hand sides at once
B = np.array([[5, 8],
              [12, 19]])   # two b vectors as columns
X = LA.solve(A, B)
print(X)   # Each column is the solution for one b vector
```

9.6.4 Eigenvalues and Eigenvectors

```
import numpy as np
from numpy import linalg as LA

A = np.array([[4, 2],
              [1, 3]])

# Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors = LA.eig(A)
print('Eigenvalues:', eigenvalues)
# [5. 2.]

print('Eigenvectors (as columns):',)
print(eigenvectors)
# [[ 0.894 -0.707]
#  [ 0.447  0.707]]

# Verify: A @ v = λ * v  for each eigenpair
for i in range(len(eigenvalues)):
    lam = eigenvalues[i]
    vec = eigenvectors[:, i]  # i-th column
    print(f'λ={lam:.1f}: A@v = {A @ vec},  λ*v = {lam * vec}')

# For symmetric matrices: use eigh() — more numerically stable
S = np.array([[4, 2],
              [2, 3]])
eigvals, eigvecs = LA.eigh(S)   # guaranteed real eigenvalues
```

Section 9.7

Reading and Writing Data

Text files, binary .npy format, and saving multiple arrays

9.7.1 np.loadtxt() and np.savetxt() — Text-based I/O

```
import numpy as np

# ── Saving to text file ──
data = np.array([[1.5, 2.7, 3.2],
                 [4.1, 5.8, 6.3],
                 [7.9, 8.0, 9.4]])

np.savetxt('data.csv', data, delimiter=',', fmt='%.2f',
           header='col1,col2,col3', comments='')

# ── Loading from text file ──
loaded = np.loadtxt('data.csv', delimiter=',', skiprows=1)
print(loaded)

# Skip comment lines (lines starting with #)
np.loadtxt('data.txt', comments='#')

# Load specific columns
np.loadtxt('data.csv', delimiter=',', skiprows=1,
           usecols=(0, 2))   # Only columns 0 and 2

# Load as specific dtype
np.loadtxt('data.csv', delimiter=',', skiprows=1, dtype=np.float32)

# genfromtxt — handles missing values
data = np.genfromtxt('data.csv', delimiter=',',
                     skip_header=1, filling_values=0.0)
```

NOTE: np.loadtxt() is strict — it fails if any value is missing. Use np.genfromtxt() when your data may have missing values (NaN) or irregular rows.

9.7.2 np.save() and np.load() — Binary .npy Format

```
import numpy as np

arr = np.array([[1.5, 2.7], [3.2, 4.1]])

# Save as binary .npy file (preserves dtype and shape)
np.save('array.npy', arr)

# Load back — exact same array (dtype, shape, values)
loaded = np.load('array.npy')
```

```
print(loaded.dtype)    # float64  — preserved!
print(loaded.shape)    # (2, 2)   — preserved!

# .npy is much faster than text for large arrays
# and takes less disk space (no string conversion)
```

> **TIP:** Use .npy format for intermediate results and checkpoints in data pipelines. It is much faster and more compact than CSV for large numerical arrays.

9.7.3 np.savez() — Saving Multiple Arrays

```
import numpy as np

X = np.array([[1, 2], [3, 4]])
y = np.array([0, 1])
metadata = np.array(['class_0', 'class_1'])

# Save multiple arrays to a .npz file (like a zip of .npy files)
np.savez('dataset.npz', features=X, labels=y, names=metadata)

# Load back
data = np.load('dataset.npz')
print(data.files)              # ['features', 'labels', 'names']
print(data['features'])        # [[1 2] [3 4]]
print(data['labels'])          # [0 1]

# Compressed version (smaller file, slightly slower)
np.savez_compressed('dataset_compressed.npz', features=X, labels=y)

# Common workflow: save train/test split
np.savez('train_test_split.npz',
         X_train=X_train, X_test=X_test,
         y_train=y_train, y_test=y_test)
# Load later:
# d = np.load('train_test_split.npz')
# X_train = d['X_train']
```

Format	When to Use	Notes
.npy	Single array, fast I/O	np.save / np.load
.npz	Multiple arrays together	np.savez / np.load
.txt / .csv	Human-readable, exchange with other tools	np.savetxt / np.loadtxt
.csv (pandas)	Complex tables with headers/types	pd.read_csv → .values

Section 9.8

Lab: NumPy in Practice

Hands-on exercises: real data, benchmarking, and linear system solving

9.8.1 Matrix Operations on a Real Dataset

In this lab exercise, we simulate loading a student marks dataset and perform common analysis operations using NumPy:

```
import numpy as np

# Simulate a dataset: 8 students, 5 subjects
# Columns: Math, Physics, Chemistry, English, Computer Science
np.random.seed(2024)
marks = np.random.randint(40, 100, size=(8, 5)).astype(float)

subjects = ['Math', 'Physics', 'Chemistry', 'English', 'CS']

print('=== Student Marks Dataset ===')
print(marks)
print()

# 1. Overall statistics
print('=== Summary Statistics ===')
print(f'Highest mark : {marks.max():.1f}')
print(f'Lowest mark  : {marks.min():.1f}')
print(f'Mean score   : {marks.mean():.2f}')
print(f'Std deviation: {marks.std():.2f}')
print()

# 2. Per-subject averages (axis=0: average down rows = column mean)
subject_avg = marks.mean(axis=0)
print('=== Subject Averages ===')
for subj, avg in zip(subjects, subject_avg):
    print(f'  {subj:12s}: {avg:.2f}')
print()

# 3. Per-student total and average (axis=1)
student_total = marks.sum(axis=1)
student_avg   = marks.mean(axis=1)
print('=== Student Performance ===')
for i, (total, avg) in enumerate(zip(student_total, student_avg)):
    grade = 'A' if avg >= 80 else ('B' if avg >= 65 else 'C')
    print(f'  Student {i+1}: Total={total:.0f}, Avg={avg:.1f}, Grade={grade}')
print()

# 4. Top scorer per subject using argmax
print('=== Top Scorer Per Subject ===')
top_scorers = marks.argmax(axis=0)   # Index of max in each column
for subj, stu in zip(subjects, top_scorers):
    print(f'  {subj}: Student {stu+1} ({marks[stu, subjects.index(subj)]:.0f})')
print()
```

```
# 5. Normalize marks to 0-1 range (per subject)
min_marks = marks.min(axis=0)    # shape (5,) — min per subject
max_marks = marks.max(axis=0)    # shape (5,) — max per subject
normalized = (marks - min_marks) / (max_marks - min_marks)
print('=== Normalized Marks (first 3 students) ===')
print(np.round(normalized[:3], 3))
```

9.8.2 Performance Benchmarking: NumPy vs Python Lists

```
import numpy as np
import time

N = 5_000_000   # 5 million elements

# ── Test 1: Element-wise multiplication ──
py_list = list(range(N))
np_arr  = np.arange(N, dtype=np.float64)

t0 = time.perf_counter()
result_py = [x * 2.5 for x in py_list]
t1 = time.perf_counter()
print(f'Python list multiply: {(t1 - t0)*1000:.1f} ms')

t0 = time.perf_counter()
result_np = np_arr * 2.5
t1 = time.perf_counter()
print(f'NumPy array multiply: {(t1 - t0)*1000:.1f} ms')

# ── Test 2: Sum ──
t0 = time.perf_counter()
total = sum(py_list)
t1 = time.perf_counter()
print(f'Python sum: {(t1 - t0)*1000:.1f} ms')

t0 = time.perf_counter()
total = np_arr.sum()
t1 = time.perf_counter()
print(f'NumPy sum:  {(t1 - t0)*1000:.1f} ms')

# ── Test 3: Dot product ──
a_list = list(range(10000))
b_list = list(range(10000))
a_arr  = np.arange(10000, dtype=np.float64)
b_arr  = np.arange(10000, dtype=np.float64)

t0 = time.perf_counter()
dot_py = sum(x * y for x, y in zip(a_list, b_list))
t1 = time.perf_counter()
print(f'Python dot product: {(t1 - t0)*1000:.3f} ms')

t0 = time.perf_counter()
dot_np = np.dot(a_arr, b_arr)
t1 = time.perf_counter()
print(f'NumPy dot product:  {(t1 - t0)*1000:.3f} ms')
```

```
# Typical output:
# Python list multiply: 380.5 ms
# NumPy array multiply:   8.2 ms   (~46x faster)
# Python sum:            82.1 ms
# NumPy sum:              3.4 ms   (~24x faster)
```

9.8.3 Solving a Linear System with np.linalg.solve

Real-world problem: A company produces three products (A, B, C). Using production constraints, find how many units of each product to manufacture.

```
import numpy as np
from numpy import linalg as LA

# Problem: Find x (units of A), y (units of B), z (units of C)
#
#  Machine 1 constraint: 2x +  y +  z = 14   (total machine hours)
#  Machine 2 constraint:  x + 3y + 2z = 18   (total machine hours)
#  Machine 3 constraint:  x +  y + 4z = 16   (total machine hours)

# Coefficient matrix A
A = np.array([[2, 1, 1],
              [1, 3, 2],
              [1, 1, 4]], dtype=float)

# Right-hand side b
b = np.array([14, 18, 16], dtype=float)

print('Coefficient Matrix A:')
print(A)
print('Right-hand side b:', b)
print()

# Check that the system has a unique solution
print(f'Determinant of A: {LA.det(A):.4f}')
# If det ≠ 0, the system has exactly one solution

# Solve the system
solution = LA.solve(A, b)
print(f'Solution: Product A = {solution[0]:.2f} units')
print(f'          Product B = {solution[1]:.2f} units')
print(f'          Product C = {solution[2]:.2f} units')

# Verify the solution
print()
print('Verification (A @ x should equal b):',
      np.allclose(A @ solution, b))
print('Residual:', np.round(A @ solution - b, 10))

# Condition number — measures numerical stability
cond = LA.cond(A)
print(f'Condition number: {cond:.4f}')
# A condition number close to 1 is ideal.
# Very large (>1e12) means the matrix is near-singular
```

> **NOTE:** np.linalg.solve(A, b) is preferred over computing inv(A) @ b. It is both faster (uses LU decomposition internally) and numerically more stable, especially for large or nearly-singular matrices.

Summary and Key Takeaways

Section	Key Concept	Essential Functions
9.1 Introduction	NumPy = fast arrays via C	import numpy as np
9.2 Creating Arrays	Shape, dtype, N-dimensional	array, zeros, ones, arange, linspace, random
9.3 Operations	Vectorized, no loops	*, +, sqrt, sum, mean, std, cumsum
9.4 Indexing	Views vs copies	arr[i], arr[mask], arr[[i,j]]
9.5 Manipulation	Shape changes + combining	reshape, stack, split, transpose
9.6 Linear Algebra	Matrix math + solving Ax=b	dot, @, linalg.solve, eig, inv
9.7 File I/O	Save and reload arrays	save, load, savetxt, loadtxt, savez
9.8 Lab	End-to-end practice	All of the above

Important Rules to Remember:

- Use @ for matrix multiplication; * is element-wise
- Slices and reshape() return views — modifying them modifies the original
- Use .copy() to avoid unintentional modification of the original
- Boolean mask conditions must use & and | with parentheses, not 'and'/'or'
- Use np.linalg.solve() instead of computing the inverse manually
- Set random seeds (np.random.seed) for reproducible results
- Choose np.linspace when you need N points; np.arange when you know the step
- Always set a dtype for performance-sensitive code (float32 vs float64)

Quick Reference

NumPy Cheat Sheet

Frequently used functions and patterns at a glance

Array Creation

```
np.array([1,2,3])           # from list
np.zeros((m, n))            # m×n zeros
np.ones((m, n))             # m×n ones
np.eye(n)                   # n×n identity
np.full((m, n), val)        # m×n filled with val
np.arange(start, stop, step)# range as array
np.linspace(start, stop, n) # n equally-spaced points
np.random.rand(m, n)        # uniform [0,1)
np.random.randn(m, n)       # standard normal
np.random.randint(lo, hi, (m,n))  # random integers
```

Array Attributes & Reshape

```
arr.shape          # tuple of dimensions
arr.ndim           # number of dimensions
arr.size           # total element count
arr.dtype          # data type
arr.reshape(m, n)  # new shape (view if possible)
arr.flatten()      # 1D copy
arr.ravel()        # 1D view
arr.T              # transpose (view)
```

Indexing

```
arr[i]             # 1D element
arr[r, c]          # 2D element
arr[1:4]           # slice (view)
arr[arr > 5]       # boolean indexing (copy)
arr[[0, 2, 4]]     # fancy indexing (copy)
arr[:, 1]          # entire column 1
arr[0, :]          # entire row 0
```

Math & Aggregation

```
np.sqrt(arr)        np.exp(arr)         np.log(arr)
np.abs(arr)         np.round(arr, n)    np.clip(arr, lo, hi)
arr.sum(axis=0)     arr.mean()          arr.std()
arr.min()           arr.max()           arr.cumsum()
arr.argmin()        arr.argmax()        np.sort(arr)
```

Linear Algebra

```
A @ B                    # matrix multiplication
np.dot(A, B)             # same as @
np.linalg.solve(A, b)   # solve Ax = b
np.linalg.inv(A)         # inverse
np.linalg.det(A)         # determinant
np.linalg.eig(A)         # eigenvalues, eigenvectors
np.linalg.norm(A)        # matrix norm
```

File I/O

```
np.save('file.npy', arr)              # save one array
np.load('file.npy')                   # load one array
np.savez('file.npz', x=arr1, y=arr2) # save multiple
data = np.load('file.npz'); data['x'] # load multiple
np.savetxt('file.csv', arr, delimiter=',')  # text
np.loadtxt('file.csv', delimiter=',')       # text
```

Vikas Karanth U at 2/21/2026 10:47 PM